

The Effect of Call-Graph Construction Algorithms on Semantic Conflict Detection

Matheus Barbosa, Paulo Borba, Rodrigo Bonifácio, Victor Lira, and Galileu Santos de Jesus

Abstract—Collaborative software development improves productivity but introduces integration challenges, such as semantic conflicts. These occur when parallel changes are textually integrated but semantically interfere. Although researchers propose static analyses to detect them, these analyses suffer from high false positive rates. To understand how different call-graph construction algorithms affect semantic conflict detection, we conduct an empirical study. We extended three existing static analyses with four call-graph algorithms, ranging from non-points-to-based (CHA, RTA) to points-to-based (SPARK, VTA). We evaluate these 12 configurations on two datasets to compare their accuracy and performance. Although pruned call graphs typically improve static analyses, their effect on semantic conflict detection reveals a trade-off: algorithms avoiding overtainting, like SPARK, reduce timeouts and false positives, improving precision. However, this aggressive pruning significantly increases false negatives, sharply reducing recall compared to more conservative algorithms. Consequently, no single configuration is universally optimal for semantic conflict detection. Researchers and developers must carefully select call-graph algorithms based on their priorities, balancing the cost of false alarms and execution time against the risk of missed conflicts.

Index Terms—merge tools, semantic conflict, static analysis, call-graph construction, points-to analysis

I. INTRODUCTION

Textual merge tools operate on a line-by-line basis and report conflicts when developers change the same or adjacent lines of code [1], [2]. More advanced semistructured and structured merge tools [3]–[5] leverage programming language syntax and static semantics, and avoid some of the drawbacks of textual tools.

Textual and structured merge tools, however, are unable to detect *dynamic semantic conflicts*, which occur when changes from different developers are textually integrated but semantically interfere with each other [6]–[8], as when two developers add new assignments to the same state element.¹ Semantic conflicts often go unnoticed and lead to unexpected behavior, as they are hard to detect through typical quality assurance practices like code review and testing [9]. This way, they pose risks in both fork-based and centralized projects [10], [11]. When detected, they are often hard to resolve, especially if discovered late.

M. Barbosa, P. Borba, R. Bonifácio, V. Lira, and G. S. de Jesus are with the Centro de Informática, Universidade Federal de Pernambuco, Recife, Brazil. E-mail: {mbo2, phmb, rbonifacio, vl, gsj}@cin.ufpe.br

¹State elements refer to program entities that hold or represent the state of a program during its execution. These include global variables, object fields, system files and streams, variables that store method return values, raised exceptions, and any other entities that maintain state information across different execution contexts.

To avoid these problems, researchers have proposed static analyses for detecting semantic conflicts [6], [7], [12], [13] at merge time, but these analyses often suffer from high false positive rates and false negatives. Such false positives result from the imprecision typically associated with static analyses, particularly the lack of edge pruning in conservative call graphs used by the analysis. While some algorithms (e.g., SPARK [14]) achieve large reductions in these graphs using points-to information [15], others (e.g., CHA [16]) use class hierarchy information and populate a much coarser structure, which can inflate the set of possible targets and over-approximate behavior.

In this paper, we evaluate the effect of call-graph construction algorithms on semantic conflict detection accuracy and performance. For this purpose, we developed aligned and extensible implementations for three existing analyses [13], supporting configurations with four call-graph algorithms ranging from points-to-based (*PA*) algorithms (e.g., VTA, SPARK) to non-points-to-based (*noPA*) algorithms (e.g., CHA, RTA). Our evaluation spans multiple merge scenarios² from two Java datasets: the first with 99 manually labeled scenarios [13] for measuring precision, recall, and F1-score, and the second with 920 unlabeled scenarios [17] for analyzing divergence and computational performance. We combine code instrumentation with manual analysis to characterize the resulting trade-offs.

Our results reveal a trade-off. Algorithms that avoid overtainting, like SPARK and VTA, effectively prune edges, almost eliminating timeouts and reducing false positives. This reduction improves precision, overall accuracy, and execution time. However, this aggressive edge pruning comes at a high cost for semantic conflict detection: it introduces false negatives, leading to large drops in recall when compared to *noPA* algorithms that maintain a more conservative call graph.

Manual inspection shows that this drop in recall is mainly caused by the aggressively pruned call graphs produced by *PA* algorithms, which struggle with dynamic Java features, such as reflection and proxy-based dispatch, and thus disconnect call paths in which semantic conflicts occur. Our findings demonstrate that employing advanced static analysis algorithms is indeed faster than using CHA. This aligns with Bodden’s observations [18] that higher precision actually improves computational performance by avoiding overtainting. **Although the advanced algorithms successfully reduce false positives and improve overall accuracy, the resulting increase in false negatives remains detrimental. Our findings align**

²A merge scenario consists of a quadruple formed by a common ancestor (Base), a merge commit, and its two parents (Left and Right).

with systematic robustness evaluations [19] indicating that this unsoundness is severely aggravated in the context of semantic conflict detection, forcing developers to decide whether the computational performance and precision gains are worth the loss in coverage.

Our results suggest that, in the context of semantic conflict detection, no single analysis configuration is universally optimal: developers and researchers must carefully select call-graph algorithms based on their specific priorities, balancing the cost of false alarms and execution time against the risk of missed conflicts.

The main contributions of this paper are as follows: (i) aligned and extensible implementations of three static analyses for semantic conflict detection, designed to support configurations with four call graph construction algorithms, including CHA, RTA, VTA, and SPARK; (ii) an empirical evaluation of the impact of call-graph construction algorithms on semantic conflict detection; (iii) evidence of the trade-offs introduced by call-graph algorithms in terms of precision, recall, and performance; and (iv) a publicly available replication package to support reproducibility.

II. MOTIVATING EXAMPLE

To illustrate the effect of call-graph algorithms on semantic conflict detection, consider the example in Figure 1. It shows Java code resulting from a merge that integrates changes independently made by two developers, Left in red and Right in blue, to a common Base commit. The remaining code corresponds to the original content in the Base commit.

```
class Text {
    private String t;
    private Report r;
    Text(Report r, String t) {
        this.r = r; this.t = t;
    }
    void generateReport() {
        r.countDupWords();
        r.countComments();
        r.countDupWhiteSpace();
    }
}
```

Fig. 1. Successful textual merge integrating the changes of two developers, one highlighted in red and the other in blue

To improve the text report, Left added a call to `r.countDupWords()`, which counts duplicate consecutive words in the text. Right independently added a call to the `r.countDupWhiteSpace()` method, which is responsible for counting the number of consecutive whitespace sequences in the text. As the changes made by Left and Right were not in the same or consecutive lines, textual merge was successful, with no report of a merge conflict.

Now consider a static analysis³ that detects semantic conflicts whenever the same state element (variable, object field,

etc.) is modified by the changes made by two contributors. Running this analysis for the code in Figure 1 reports a conflict, given that an instance of `ReportSimple` (see Figure 2) might be the target of the calls `r.countDupWords` and `r.countDupWhiteSpace`. Note that both methods write to the same field `fixes`. On the other hand, we can assume that the call to `countComments` (omitted from the example for simplicity) does not contribute to the conflict because it writes to a different field. For now, consider that `ReportSimple` is the only implementation of the `Report` interface.

For instance, generating a report for the `"the_the_dog_dog"` text results in `fixes` being 1, ignoring duplicate word counting as the second assignment to `fixes` overrides the first. Note that, in the only Left branch, the execution would result in `fixes` being 2.

In this case, we say that **Right interferes with Left**, as it breaks the change made by Left.

```
class ReportSimple implements Report {
    void countDupWords() { ...
        fixes++; ...
    }
    void countDupWhiteSpace() { ...
        fixes = count; ...
    }
}
```

Fig. 2. ReportSimple implementation

```
class ReportAdvanced implements Report {
    void countDupWords() { ...
        dupWords++; ...
    }
    void countDupWhiteSpace() { ...
        dupWhiteSpace++; ...
    }
}
```

Fig. 3. ReportAdvanced implementation

Consider now that one provides another implementation of the `Report` interface, say `ReportAdvanced` (see Figure 3), which counts duplicated words and whitespaces separately, storing them in different fields. In such a situation, an analysis configuration that doesn't employ *PA* algorithms reports a conflict, as it assumes that both implementations of `Report` can be invoked at runtime, independently of how `Text` is initialized. A *PA* analysis configuration, however, correctly reports a conflict only if it collects information that `Text` is initialized with a `ReportSimple` instance; note that the methods in `ReportAdvanced` don't write to the same state element. If `Text` is initialized with a `ReportAdvanced` instance, *noPA* incorrectly reports a conflict (a false positive).

This example illustrates how the use of variations of a call-graph algorithm might improve the refinement of static

³We describe this analysis in more detail in the following section.

analysis for conflict detection. In this case, greater refinement is achieved because algorithms like *SPARK* allow for the construction of a more aggressively pruned call graph than the one used by *noPA* algorithms. Refinement can also be improved by how *PA* stores information on the analysis data structures, as discussed later in the paper. These refinements, though, come with drawbacks. For example, the scenario discussed previously relies on a `main` method (not shown in the figures) that explicitly instantiates the objects using the `new` command. If this `main` method used dynamic features of the Java programming language—such as reading the class to be instantiated from a properties file and using reflection to create the object—*PA* algorithms might miss actual conflicts. These cases often lead to false negatives. Because of the involved trade-offs, opting for *PA* algorithms instead of *noPA* algorithms might not always be the best choice. Therefore, it is important to rigorously assess the benefits and drawbacks of using different call-graph algorithms in the context of semantic conflict detection.

III. STATIC ANALYSIS FOR SEMANTIC CONFLICT DETECTION

In a previous work, we explored a set of static analyses to detect semantic conflicts [13]. We run these analyses in the merged version of the code, which is annotated with metadata indicating instructions modified or added by each developer that contributed to the merge. The analyses try to explore potential conflicting situations by keeping track of the changes developers make and how they affect *state elements* such as global variables, object fields, and variables that hold method return values.

For assessing the impact of call-graph construction algorithms on semantic conflict detection, we focus on three interprocedural analyses [13]:

- `idf` Interprocedural Direct Flow, which captures def–use relationships involving Left and Right contributions as data flow, including edges for expressions used in control structures and return statements.
- `icf` Interprocedural Confluence Flow, which accounts for interference, when both contribute data flows to a common statement.
- `ioa` Interprocedural Override Assignment, which captures situations in which state updates introduced by one developer are overridden by updates from another, as illustrated in the motivating example.

These analyses capture complementary forms of semantic interference between contributions. The choice to focus on this specific set of analyses was deliberate, as it allows for a controlled investigation of how reducing overtainting by *PA* impacts conflict detection mechanisms that rely on call-graph information and interprocedural propagation. These analyses share core characteristics with a broader class of semantic analyses, particularly their reliance on call-graph construction and interprocedural abstractions, which are precisely the aspects affected by the algorithm’s overtainting reduction capabilities.

For a rigorous assessment of the effect of call-graph algorithms on semantic conflict detection, we extend existing

analyses to support the selection of different call graphs, enabling the use of alternative construction strategies to model method call dependencies. We use version 4.5.0 of the Soot framework [20] as the underlying infrastructure for program analysis.

To ensure that our findings generalize across different call-graph construction strategies, we evaluate the analysis using multiple call-graph algorithms provided by the Soot framework: *CHA* (Class Hierarchy Analysis) [16], *RTA* (Rapid Type Analysis) [21], *VTA* (Variable Type Analysis) [22], and *SPARK* [14]. These algorithms span a broad spectrum of edge pruning capabilities, ranging from the coarse-grained, conservative call-graph approximation of *CHA* to the pruned, points-to-driven model of *SPARK*. This allows us to capture the influence of call-graph construction under diverse characteristics, from coarse-grained approximations to refined models.

As *CHA* relies solely on the class hierarchy to resolve virtual calls, it systematically assumes that any subclass implementation may serve as a potential target. It tends to be conservative. As a consequence, it can generate false positives by erroneously classifying assignments as conflicting when, in reality, they involve writes to state elements residing in different memory regions, as illustrated in the motivating example.

We also experiment with two intermediate call-graph construction strategies provided by Soot: *RTA* and *VTA*. *RTA* refines *CHA* by considering only the types of classes that are actually instantiated in the program. This reduces part of *CHA*’s over-approximation but still lacks detailed object-level distinctions, often yielding moderately conservative call graphs. *VTA* goes further by propagating type information across variables, enabling a more fine-grained understanding of potential receiver types. This typically results in a more pruned call graph than *RTA*, though still less pruned than more comprehensive points-to analyses like *SPARK*.

In contrast, *SPARK* performs a field-sensitive but context-insensitive points-to analysis. It provides more edge pruning on virtual method call dependencies by computing points-to sets, improving the accuracy of method-call resolution and reducing the likelihood of spurious conflicts.

In our experiments, we used all call graph algorithms with their default settings, which allows us to isolate the impact itself, rather than confusing it with configuration-specific optimizations.

Our analyses operate with two distinct types of entry points. First, in both datasets, the analysis entry points correspond precisely to the methods that were concurrently modified by two developers; the analysis begins from each such method and explores its reachable behavior. Second, the call-graph entry points determine the roots used by Soot when constructing the call graph. Since objects can be created outside the scope of the analysis entry point, we use all main methods in a project as entry points for the call graph. For projects without main methods, like libraries, we conservatively use all public methods as call graph entry points.

The analyses traverse code recursively using the specific call graphs and apply filtering strategies to ignore irrelevant

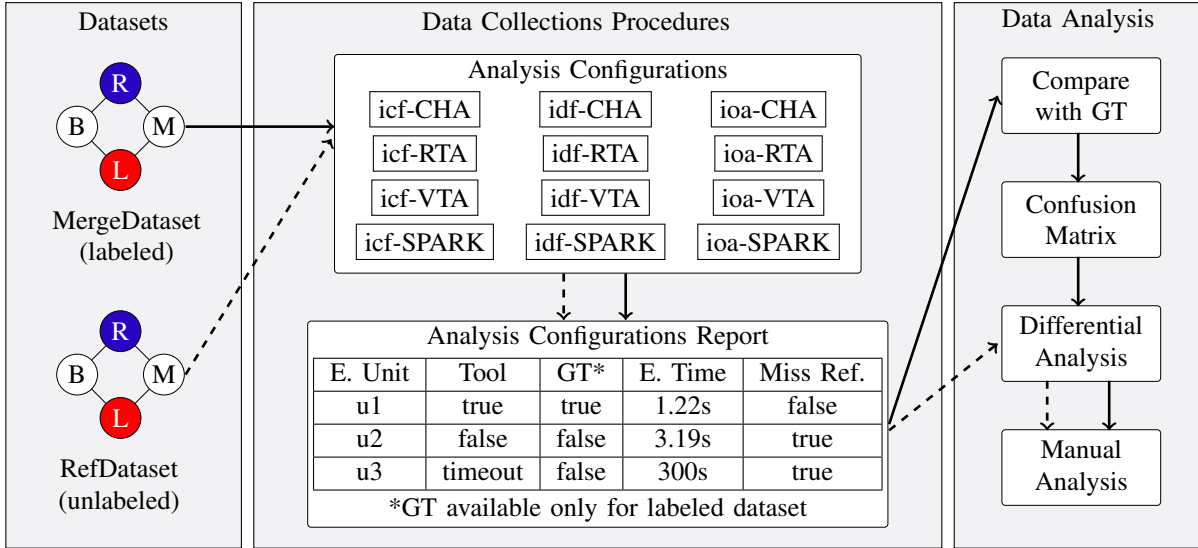


Fig. 4. Study settings workflow

or unsupported methods. In the end, reports and diagnostic logs are generated to support further inspection.

IV. STUDY SETTINGS

To investigate the effect of call-graph construction algorithms on semantic conflict detection, we conduct an empirical study comparing them in terms of accuracy and computational performance.

We pay attention to the reduction of false positives (FPs), potential changes in false negatives (FNs), and the frequency in which analysis configurations differ. With this goal, we formulate the following research questions:

- (RQ1) How do different call-graph algorithms affect conflict detection accuracy?
- (RQ2) What is the magnitude and nature of the discrepancies between different call-graph algorithms?
- (RQ3) How do different call-graph algorithms affect the computational performance of the analysis?

These questions allow us to understand the effect of call-graph construction algorithms and provide valuable insights into how to choose the analysis configuration in practice. This understanding helps practitioners and researchers make informed decisions about trade-offs between accuracy and execution time, ensuring a more effective and efficient analysis.

To answer these research questions, we follow the experimental design illustrated in Figure 4 and explained in the rest of this section.

A. Datasets

To answer our research questions, we use merge scenarios from two Java datasets, MergeDataset [13] and RefDataset [17], which are respectively illustrated in the top and bottom part of the left panel of Figure 4. The first dataset is, to the best of our knowledge, the largest dataset labeled with semantic-conflict ground truth, which is considerably hard to establish manually, as it often requires a deep semantic understanding of

the merged code. The other is unlabeled, but is much larger and diverse, which allows us to better understand the differences between the analyses.

So, we use the MergeDataset to answer the first research question RQ1 and evaluate the accuracy of the proposed analyses to detect semantic conflicts. It contains merge scenarios in which two contributors changed the same methods or constructors, and comprises a manually labeled set of 99 experimental unities associated with 54 merge scenarios from 32 open-source Java systems. As a merge scenario might have more than one declaration that was independently changed by both developers, we consider pairs (scenario, declaration) as our experimental units. The MergeDataset scenarios [7]–[9], [13]⁴ have been used in previous studies on semantic conflicts.

To answer both of our research questions RQ2 and RQ3 and understand the magnitude and nature of the discrepancies between the analysis configurations, as well as their differences in computational performance, we additionally use the RefDataset [17].

Similarly to the MergeDataset, the RefDataset contains methods or constructors modified by two contributors. The RefDataset⁵ consists of 920 experimental units from 295 projects, selected and balanced satisfying the following criteria: GitHub’s greatest hits repositories with at least 10 stars, using Maven or Gradle for build automation, and with the latest main branch commit successfully compiling and passing tests within 30 minutes on JDK 8, 11, or 17. This ensures a higher likelihood of automated .jar file generation, which is needed for running the analyses. Lastly, we applied an additional filter to confirm that the target class and method changed by two developers were included in the final build’s .jar file. This large dataset allows us to better evaluate the differences caused by varying call-graph edge pruning capabilities in more varied and less restrictive scenarios where the expected results of the analysis are not previously known.

⁴<https://github.com/spgroup/mergedataset>

⁵<https://github.com/spgroup/ref-dataset>

For each scenario in both datasets, we obtain the JAR file of the merge commit version without external dependencies. This reduces the amount of code analyzed, improving execution time, although it may decrease accuracy; this trade-off is promising for practical uses of static analysis in detecting interference, and thus we adopt this design. Along with this, we collect the line numbers corresponding to the changes introduced by the Left and Right sides. We focus on cases where two developers modified the same method, as such scenarios are more prone to behavioral interference.

B. Data Collections Procedures

With the prepared inputs, we run the twelve analysis configurations (3 analyses x 4 call graph algorithms) for each experimental unit in both datasets. To ensure practical execution times, we control the depth and duration of our analyses. We limit the analysis depth to five levels of method calls from the entry point. Preliminary experiments show this depth preserves accuracy while keeping execution times feasible. **We also set a 300-second (five-minute) timeout per analysis. We base this threshold on empirical continuous integration data showing that successful builds complete in five minutes on median [23]. Prolonging execution beyond this limit strongly correlates with broken builds, as waiting for long-running tasks rarely prevents failures. Setting this timeout also ensures the feasibility of running the experiment at scale without artificially inflating costs. If an analysis exceeds this limit, we interrupt it and mark it with a timeout status.**

We executed our experiment with default settings for the configuration of the Jimple Body Creation (jbc) phase of Soot, except for the (use-original-names) option, which preserves the original names of local variables present in the source code. We activate this to make the conflict reports generated by the analyses more readable and informative, so that users can rely only on source code information to interpret them. We also configure Soot so that it keeps line numbers from bytecode, and doesn't analyze classes in standard Java libraries, except for basic classes like `String`, `RuntimeException`, and the ones that simply encapsulate primitive types (`Integer`, `Long`, etc.). So classes like `ArrayList` and `HashMap` are not explored by our analysis, for example. Just as the builds of the scenarios used in our datasets do not have external dependencies, we also reduce this way the amount of code to be analyzed, to focus on developer changed code.

To avoid variations in software and hardware configuration, we create a `Docker` file that organizes all the necessary experimental infrastructure. This, together with the repositories and detailed instructions for running the experiment, helps repeatability and reproducibility.

To control for execution-time variation, we run each analysis configuration 10 times for every experimental unit. We execute all experiments inside `Docker` containers, distributing runs across 10 parallel container instances. We configure each container to use Linux Ubuntu with a memory limit of 20 GB and 4 dedicated CPU cores, using `Docker` resource constraints to ensure isolation, reproducibility, and consistent execution conditions while reducing overall experimental runtime.

For evaluation purposes, the result of an analysis is considered positive for a specific experimental unit if the execution reports one or more interference (this can happen when the same conflicting pair affects more than one code location, e.g., when a shared variable is accessed in different contexts within the same method).

C. Data Analysis

To answer RQ1, we compare the execution results yielded by the twelve analysis configurations for each of the 99 experimental units from the MergeDataset. The results are compared against the conflict ground truth provided by the dataset. We then compute a confusion matrix and a set of accuracy metrics, including precision, recall, accuracy, and F1-score, enabling a quantitative assessment of the analysis configurations in relation to the established ground truth.

To answer RQ2, we analyze data from the execution of our experiment using both the MergeDataset and RefDataset datasets. Unlike MergeDataset, RefDataset is not labeled, which prevents direct comparison against a ground truth. However, RefDataset is larger and more diverse, encompassing a broader range of scenarios. This diversity enhances the reliability and generalizability of our findings when analyzing differences between the analysis configurations.

Our objective is to determine how frequently each call-graph algorithm leads to discrepant outcomes, thereby quantifying the magnitude of the effect that the algorithm choice has on the final results. This analysis seeks to reveal whether selecting a more refined call-graph algorithm produces marked differences in conflict detection or if the impact is marginal. To support this comparison, we apply descriptive statistics to compute relative frequencies, construct summary tables, and analyze the observed discrepancies.

In addition, we conduct a manual inspection of representative cases to understand the underlying causes of the discrepancies. We focus on SPARK as the primary point of comparison because it is the most refined of the four algorithms studied. This allows us to assess the trade-off between the precision of the most sophisticated algorithm and the risk of missing conflicts. To support this qualitative assessment, we instrument the implementations of the three analyses with additional data. This allows us to extract insights into how the choice of algorithm influences the outcomes. In particular, our infrastructure tracks situations in which an analysis fails to resolve a reference (classified as a miss-reference⁶). This is suggestive of a conflict that is not reported due to *PA* unsoundness issues.

We present a table summarizing the results. Additionally, we compute a count of how often such miss-reference occurs in execution paths that would lead to conflicts. This helps reveal specific patterns that suppress potential conflict reports, not necessarily due to correctness, but due to excessive edge pruning.

⁶This may occur when the points-to set of a variable or reference expression is empty or undefined at the time it is accessed, such as in the case of field or array dereferences. It may also happen when the call graph fails to resolve the target method of a virtual or interface invocation.

Finally, to answer RQ3, we analyze execution time by running each analysis configuration ten times for every experimental unit in both datasets. We then apply the Shapiro-Wilk test to assess normality, followed by the Wilcoxon signed-rank test to evaluate statistical significance. In addition to these tests, we compute descriptive statistics, plot the distributions, and perform a scenario-by-scenario comparison to determine which analysis configuration performs faster for each experimental unit.

V. RESULTS

This section presents the main results of our empirical assessment. We begin with the results comparing the call-graph algorithms in terms of conflict detection accuracy. Then, in Section V-B, we provide a broader assessment of the changes in the number of reported semantic conflicts, based on both datasets. Finally, Section V-C presents the computational performance results. The complete set of results, along with step-by-step instructions for reproducing the experiments, is available in the online appendix [24].

A. RQ1: How do different call-graph algorithms compare in terms of conflict detection accuracy?

Table I presents the results obtained by the analysis configurations we discussed in Section III, using the MergeDataset.

TABLE I
OUTCOME DISTRIBUTION FOR EACH ANALYSIS CONFIGURATION USING MERGEDATASET.

Analysis configuration	True	False	Timeout
icf-CHA	47	46	6
icf-RTA	44	32	23
icf-VTA	47	49	3
icf-SPARK	47	52	0
ioa-CHA	45	45	9
ioa-RTA	37	41	21
ioa-VTA	25	71	3
ioa-SPARK	20	79	0
idf-CHA	28	68	3
idf-RTA	27	51	21
idf-VTA	25	72	2
idf-SPARK	23	76	0

Table I reveals three key patterns in the outcome distribution. First, SPARK-based configurations present the fewest timeouts. Second, VTA-based configurations yield results close to SPARK, with few timeouts, since the *PA* builds a reduced call graph, with fewer nodes and edges. In contrast, RTA configurations show the highest number of timeouts across all twelve analysis configurations, suggesting that its intermediate granularity does not adequately scale for the analyzed interactions. Regarding the analyses, the *icf* analysis produces the highest number of reported conflicts (*true* results), preserving the same overall trend: RTA presents the most timeout exhaustion, while SPARK and VTA generally minimize it. These timeouts stem from two compounding factors. First, constructing the call graph itself can consume the entire time budget. Although RTA is often assumed to be lightweight, it must resolve virtual calls against all instantiated types, which in projects with deep or broad class hierarchies

can make graph construction alone prohibitively expensive, in some cases exhausting the 300 second budget before the conflict detection analysis even begins. Karakaya et al. [25] report a similar pattern: RTA spends a disproportionately large fraction of total analysis time on call-graph construction compared to CHA. Second, when construction does complete within budget, the resulting call graph retains spurious edges from overtainting, which increases the branching factor at polymorphic call sites and creates a combinatorial explosion of paths for the subsequent analysis to explore.

1) *Accuracy Assessment*: When evaluating accuracy, we distinguish *true timeouts* (the analysis times out but has already found a conflict) from *false timeouts* (the analysis times out without finding any conflict). We classify true timeouts as *true* outcomes because they successfully indicate interference. However, false timeouts provide no definitive classification. To prevent artificial bias, we exclude false timeouts from the evaluated sample. Consequently, we omit RTA from the subsequent accuracy assessment because all its timeouts are false timeouts. Including RTA would drastically reduce the comparative sample size across algorithms.

Table II presents the confusion matrix for the CHA, VTA, and SPARK configurations, and Table III details their accuracy metrics.

TABLE II
CLASSIFICATION SUMMARY FOR CHA, VTA, AND SPARK CONFIGURATIONS.

Analysis configuration	TP	FP	TN	FN	N
icf-CHA	10	40	27	19	96
icf-VTA	10	38	29	19	96
icf-SPARK	10	36	31	19	96
ioa-CHA	15	30	31	14	90
ioa-VTA	12	11	50	17	90
ioa-SPARK	5	10	51	24	90
idf-CHA	8	20	47	21	96
idf-VTA	7	18	49	22	96
idf-SPARK	7	16	51	22	96

Table II reveals a consistent trade-off: as edge pruning increases, false positives decrease while false negatives often increase. Across all analyses, false positives (FP) decrease from CHA to SPARK because a more refined call graph eliminates spurious method-call targets. For instance, *ioa* FP drops from 30 (CHA) to 11 (VTA) and 10 (SPARK). However, aggressive edge pruning causes the analysis to miss legitimate conflicts, increasing false negatives (FN). In *ioa*, FN rises from 14 (CHA) to 24 (SPARK). The *icf* and *idf* analyses exhibit a similar FP reduction, but their FN counts remain stable. The column *N* shows the effective sample size after excluding false timeouts, reducing the *ioa* sample from 99 to 90 due to CHA timeouts.

Table III quantifies the impact of these trade-offs. For *icf* and *idf*, upgrading from CHA to SPARK produces only marginal improvements. For example, *icf*-SPARK slightly improves accuracy over *icf*-CHA (0.39 to 0.43) while recall remains unchanged. However, for *ioa*, VTA significantly outperforms the alternatives. By effectively balancing true and false positives, *ioa*-VTA achieves the highest precision (0.52), accuracy (0.69), and F1 Score (0.46). Conversely,

TABLE III
ACCURACY METRICS RESULTS FOR CHA, VTA, AND SPARK
CONFIGURATIONS.

Analysis configuration	Precision	Recall	Accuracy	F1 Score
icf-CHA	0.20	0.34	0.39	0.25
icf-VTA	0.21	0.34	0.41	0.26
icf-SPARK	0.22	0.34	0.43	0.27
ioa-CHA	0.33	0.52	0.51	0.41
ioa-VTA	0.52	0.41	0.69	0.46
ioa-SPARK	0.33	0.17	0.62	0.23
idf-CHA	0.29	0.28	0.57	0.28
idf-VTA	0.28	0.24	0.58	0.26
idf-SPARK	0.30	0.24	0.60	0.27

applying the most refined algorithm (SPARK) to *ioa* overly restricts the call graph. This aggressive pruning drops recall to 0.17 and misses legitimate conflicts, demonstrating that excessive edge pruning harms this specific analysis.

Regarding the RTA algorithm, its intermediate granularity creates a significant performance bottleneck. As Table I shows, RTA triggers up to 23 timeouts per analysis. All of these are false timeouts, meaning the analysis exhausts its entire time budget without finding any conflict. If we evaluate RTA solely on the severely reduced sample of cases it actually finishes, its precision and accuracy appear competitive with CHA. However, this yields an artificial advantage by systematically discarding the hardest scenarios. In practice, RTA’s inability to scale makes it unsuitable for detecting semantic conflicts.

It is important to emphasize that the metric values reported in the table are presented per analysis configuration, and the three evaluated analyses are not the only existing analyses used to detect semantic conflicts. Consequently, many categories of conflicts cannot be detected by *icf*, *ioa*, or *idf* in isolation, regardless of the call-graph algorithm. Therefore, rather than interpreting the absolute metric values in isolation, the primary focus should be on the relative differences observed between the call-graph algorithms.

RQ1

Call-graph edge pruning consistently reduces false positives across all three analyses, but at the cost of increased false negatives, particularly for the *ioa* analysis (FN rises from 14 to 24 between CHA and SPARK). VTA achieves the best trade-off for *ioa*, with the highest precision (0.52), accuracy (0.69), and F1 Score (0.46). SPARK eliminates all timeouts but over-prunes the call graph for *ioa*, dropping recall to 0.17. For *icf* and *idf*, the differences across algorithms are marginal. RTA is unsuitable for semantic conflict detection, as all its timeouts are false timeouts caused by excessive call-graph construction cost.

B. RQ2: What is the magnitude and nature of the discrepancies between different call-graph algorithms?

To directly address the research question, the magnitude and nature of discrepancies between call-graph algorithms depend on the analysis. Focusing on the RefDataset, we observe that

for the *idf* and *icf* analyses, the divergences are relatively moderate, ranging from 1.20% (for *icf*-VTA) to 11.30% (for *idf*-RTA) when comparing baseline algorithms to SPARK. In contrast, the *ioa* analysis is much more sensitive, with divergences reaching up to 29.24% (comparing CHA to SPARK). Crucially, McNemar’s test confirms that the discrepancies between SPARK and *every* baseline configuration (CHA, RTA, and VTA) are statistically significant ($p \leq 0.0159$) across all three analyses, reflecting consistent behavioral differences within the RefDataset regardless of the analysis type. We focus comparisons against SPARK as it represents the extreme of edge pruning, highlighting the sharpest trade-offs. Table IV summarizes these outcomes for RefDataset.

TABLE IV
RESULTS REPORTED BY THE ANALYSIS CONFIGURATIONS USING
REFDATASET

Analysis configuration	True	False	Timeout
icf-CHA	520	356	44
icf-RTA	505	325	90
icf-VTA	509	372	39
icf-SPARK	516	381	23
ioa-CHA	513	375	32
ioa-RTA	513	376	31
ioa-VTA	297	597	26
ioa-SPARK	274	644	2
idf-CHA	238	645	37
idf-RTA	261	579	80
idf-VTA	233	653	34
idf-SPARK	226	683	11

The effect of SPARK includes a reduction in timeouts for all three analyses (*icf*, *ioa*, and *idf*), suggesting that its edge pruning capabilities generate a smaller graph by reducing overtainting. For instance, SPARK reduced timeouts in the *ioa* analysis by approximately 94% compared to CHA, converting these previously indefinite cases into definitive results. Comparing the three analyses, *ioa* exhibited the highest sensitivity to the call-graph algorithm, showing the largest shift in result distribution and the highest divergence rate.

Figure 5 details the nature of the divergences between the baseline analysis configurations and their SPARK counterparts. In the notation $A \rightarrow B$, A is the outcome produced by the baseline analysis configuration (using the CHA, RTA, or VTA algorithm) and B is the outcome produced by its SPARK counterpart; for example, $\text{True} \rightarrow \text{False}$ means the baseline analysis configuration reports a semantic conflict while the SPARK counterpart reports no conflict.

The data confirms that the discrepancies follow two distinct patterns: timeout resolution and conservative call-graph edge pruning. The blue and brown segments demonstrate that a large portion of the disagreements across all analyses are cases where SPARK resolved a baseline timeout into a definitive result. For instance, in the *icf*-RTA configuration, the combination of $\text{Timeout} \rightarrow \text{False}$ and $\text{Timeout} \rightarrow \text{True}$ accounts for 67 divergences. Additionally, the prominent green segments ($\text{True} \rightarrow \text{False}$) across all analysis configurations indicate that SPARK often refutes spurious positive results caused by overtainting in coarser abstractions. Conversely, the absence

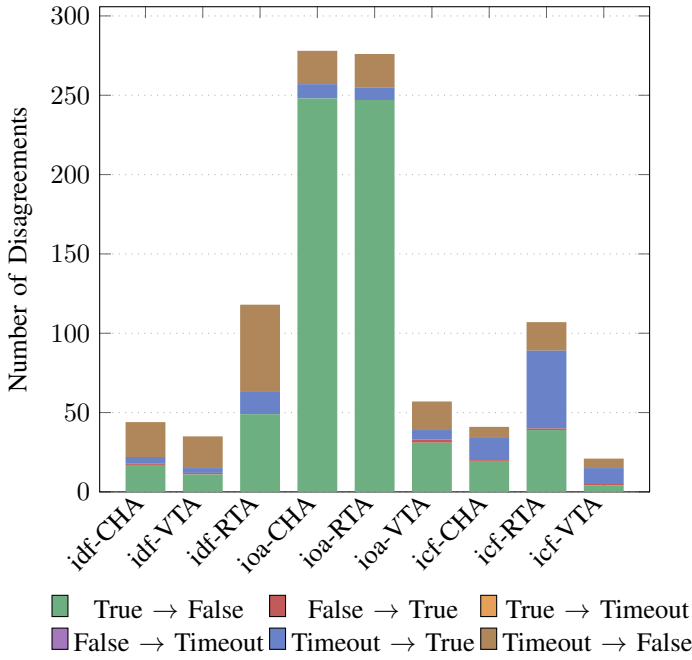


Fig. 5. Detailed Breakdown of Divergences

of orange and purple bars confirms that switching to SPARK does not introduce new performance bottlenecks.

Manual analysis of different cases: While PA reduces false positives and timeouts, it impacts recall by missing conflicts detected by simpler algorithms. These missed conflicts are especially interesting because algorithms with aggressive edge pruning theoretically prune the conservative call graph without harming recall. Understanding these missed conflicts guides improvements in call-graph algorithms and helps reduce false negatives.

To understand these discrepancies, we instrument the analysis to report instances of miss-reference (see Section IV-C), which occur when an algorithm lacks sufficient information to resolve *allocation sites* or method targets.

In the MergeDataset dataset, the ioa configuration is the most affected, with SPARK missing up to ten cases that CHA correctly detects. The impact is less severe in other configurations, where SPARK misses at most two cases in idf and icf compared to RTA. Focusing on the most affected reference scenario (ioa-CHA), we observe that four of the ten cases missed by SPARK involve at least one miss-reference on the conflict path; this missing information likely causes SPARK to prune valid paths and miss conflicts detected by the other algorithms.

Manual inspection confirms that these failures stem from dynamic language mechanisms rather than direct Java new commands, aligning with known unsoundness challenges for Java static analysis [26], [27].

Figure 6 shows a concrete instance of this phenomenon, illustrating the use of reflection that affects two experimental units of the retrofit project in MergeDataset. This example represents a recurring pattern in the codebase, where reflective and environment-dependent behavior directly impacts the

```
public class RestAdapter {

    private Response logAndReplaceResponse
        ↪ (...) throws IOException {...
        if (...) {
            for (...) {
                log.log(header.toString());
            }
        }...
        log.log(new String(bodyBytes,
            ↪ bodyCharset));...
    }

    public final class Platform {
        private static final Platform PLATFORM =
            ↪ findPlatform();

        ...
        private static Platform findPlatform() {
            try {
                Class.forName("android.os.Build");
                if (Build.VERSION.SDK_INT != 0) {
                    return new Android();
                }
            } catch (ClassNotFoundException ignored
                ↪ ) {}
            return new Base();
        }
    }
}
```

Fig. 6. Annotated code fragments with highlighted regions representing modified statements and a reflective mechanism used to identify the runtime platform implementation, illustrating dynamic behavior within the logging infrastructure.

refinement of static call-graph construction.

The unsoundness in SPARK stems from Retrofit’s runtime selection of the concrete Log implementation. For example, while CHA conservatively includes all Log implementations, SPARK depends on points-to data for its pruning. Because the logger is chosen dynamically in Platform.get() via Class.forName (brown in Figure 6), SPARK cannot infer that RestAdapter.log may reference AndroidLog. Consequently, the call log.log(...) remains unresolved, its call graph edges are missing, and SPARK cannot reach the conflicting path inside AndroidLog.log, which CHA successfully reports. Since Log is a Retrofit-defined interface, developers should not dismiss such conflicts as library-only noise.

Although the RefDataset dataset lacks labels, we extract insights by analyzing cases where a baseline algorithm reports a conflict true while SPARK refutes it false. The data shows that the ioa analysis is the most sensitive to changing the call-graph algorithm, with True → False disagreements ranging from 248 cases (CHA) to 31 cases (VTA). idf and icf are less affected, peaking at 49 (RTA) and 39 (RTA) divergences, respectively. To objectively assess the impact of

miss-references across the dataset, we focus on the `ioa-CHA` configuration. We select this configuration because `ioa` exhibits the highest sensitivity to the call-graph algorithm, and `CHA` serves as the most permissive baseline, thereby producing the largest set of divergences to investigate this phenomenon. A closer inspection reveals that in 175 of these cases, at least one miss-reference appears directly along paths related to a `CHA`-reported conflict, supporting the hypothesis that these represent false negatives introduced by the strict constraints of `SPARK`. Conversely, no miss-reference appears on the conflict paths in the remaining 73 cases, suggesting that `SPARK` correctly prunes these paths to eliminate false positives and reduce noise. Although ground truth is needed for definitive confirmation, these findings highlight a clear trade-off: while `SPARK` successfully reduces noise, its inability to resolve dynamic features inadvertently introduces false negatives in $\approx 70\%$ of the divergences.

RQ2

The magnitude of discrepancies between call-graph algorithms depends on the chosen analysis, ranging from moderate divergences in `idf` and `icf` (1.20%–11.30%) to highly sensitive shifts in `ioa` (up to 29.24%). Despite this variance, their underlying nature reveals a consistent behavioral trade-off. While `SPARK` successfully resolves timeouts (e.g., a 94% reduction in `ioa`) and eliminates false positives inherent to conservative baselines, its strict points-to constraints inadvertently introduce a substantial number of false negatives from pruning valid conflict paths when encountering dynamic behaviors. These intrinsic limitations in statically analyzing dynamic features manifest consistently and independently of the chosen analysis configuration or dataset.

C. RQ3: How do different call-graph algorithms affect the computational performance of the analyses?

To assess how the four call-graph algorithms affect the computational performance of the three analyses, we measure the average execution time of the twelve analysis configurations per experimental unit across the 10 executions. Figure 7 plots the distribution of the execution times for the RefDataset dataset.

The `SPARK` algorithm consistently exhibits the lowest average absolute execution time across all three analyses (`icf`, `idf`, and `ioa`). The non-parametric Kruskal-Wallis test reveals statistically significant differences among the four algorithms for all three analyses ($p < 0.001$). **For all three analyses, the `SPARK` algorithm is significantly faster than all other algorithms ($p < 0.0083$, with Bonferroni correction), outperforming `CHA`, `RTA`, and `VTA`.** `RTA` proves to be the most computationally expensive algorithm. **`SPARK` presents the lowest absolute mean execution times for `icf`, `idf`, and `ioa` analyses (17.9s, 24.9s, and 8.0s, respectively).**

In summary, the prevailing intuition revealed by the superior performance of the `SPARK` algorithm suggests that

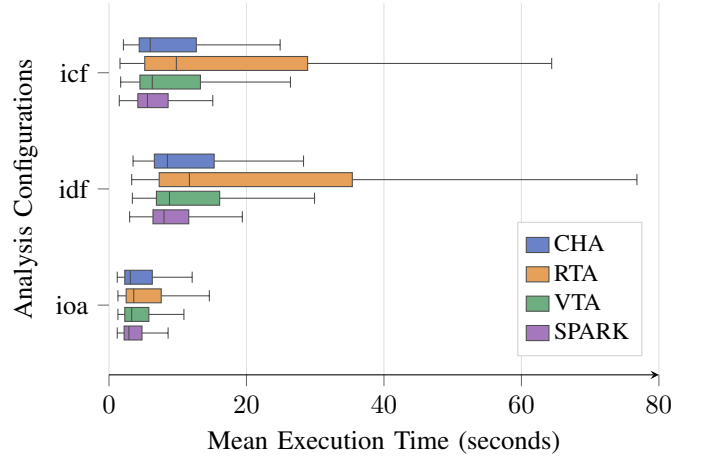


Fig. 7. Global Absolute Execution Time Comparison

the initial investment in a more aggressively pruned points-to analysis yields a leaner call graph, reducing overtainting. This increased call-graph quality acts as an optimizer for the three subsequent analyses (`icf`, `idf`, and `ioa`), which then operate over a reduced search space, processing fewer infeasible paths. In contrast, algorithms such as `RTA` and `CHA`, although theoretically lighter during graph construction [25], result in overtainting, effectively shifting the computational burden to the subsequent analyses, which waste resources traversing paths that do not exist at runtime. Therefore, within this experimental context, the edge pruning capabilities of the `SPARK` algorithm act not as a cost, but as a mechanism for overall efficiency.

RQ3

Counterintuitively, `SPARK`, although theoretically more expensive, exhibits the lowest average execution time. The computational investment required to construct a leaner call graph acts as a performance optimizer for subsequent analyses. By reducing the search space, it prevents conflict detection analyses from wasting time traversing the infeasible paths that result from overtainting in simpler algorithms like `RTA` or `CHA`.

D. Threats to Validity

Construct Validity. Although we focus on a specific set of analyses (`idf`, `icf`, and `ioa`), our results do not necessarily generalize to all forms of static analysis used for conflict detection. Analyses with different abstractions may exhibit different behaviors. However, we do not anticipate analysis-specific properties within the broader class of semantic analyses sharing similar interprocedural and pointer-sensitive foundations that would systematically lead to qualitatively different effects from those observed in our experiments. As a result, the observed effects likely extend to other analyses with similar characteristics.

External Validity. The datasets used in our study contain scenarios in which different developers concurrently modify

the same Java method. This particular case of merge scenarios may not represent the entirety of situations encountered in real-world projects, limiting the generalizability of our findings. Projects with more modular code structures, different programming languages, or distinct integration processes may exhibit conflict patterns different from those analyzed. The size of the MergeDataset limits the generalizability of our results, but to our knowledge it is the largest dataset with semantic conflict ground truth, which is hard to establish. The RefDataset partly compensates for this size limitation; it provides a large-scale collection of buildable merge scenarios derived from recent research [17], which is necessary for running static analyses.

Internal Validity. Finally, like any software system, our implementation and experimental infrastructure may contain imperfections despite our best efforts. We took great care in the development process and conducted rigorous tests to ensure the correctness and reliability of the code. While potential limitations in the implementation of pointer analysis or detection algorithms could influence the results, we are confident that these do not undermine the overall validity of our findings. Nonetheless, ongoing validation and refinement remain important to further strengthen the techniques and their application.

VI. IMPLICATIONS

Our results indicate that call-graph construction algorithms determine the effectiveness of semantic conflict detection. The choice of algorithm consistently impacts the `icf`, `idf`, and `ioa` analyses. Refined algorithms like SPARK optimize performance by reducing overtainting, but sacrifice completeness. This consistency across all three analyses suggests that call-graph pruning drives both performance gains and coverage losses.

We provide quantitative evidence of this trade-off. In RefDataset, SPARK reduces timeout cases by 94% for `ioa` and improves execution time. This shows that the reduced search space offsets the initial cost of points-to set calculation. However, the MergeDataset dataset reveals that this stability harms recall. While avoiding overtainting eliminates false positives and increases accuracy by 18% (`ioa-VTA`), aggressive edge pruning discards paths containing real interference. Consequently, configurations like `ioa-SPARK` exhibit a sharp increase in false negatives. Pruned algorithms struggle with dynamic language features, missing real conflicts that conservative algorithms like CHA successfully identify.

The gap between SPARK's efficiency and its loss of coverage reveals that "quick completion" results from ignoring hard-to-resolve execution paths. This aligns with known limitations of pointer analysis regarding complex object relationships and dynamic mechanisms like reflection and proxies. Our manual inspection confirms that PA fails to track reflection-based instantiations, directly causing the omission of real conflicts in MergeDataset.

These trade-offs align with Bodden's observation [18] that practitioners favor tools reporting real issues quickly and precisely, over fully sound approaches that lack efficiency. Thus, prioritizing fast and precise analyses is a pragmatic

choice, despite some unsoundness. Our findings show no single call-graph configuration proves universally optimal for semantic conflict detection. Algorithms minimizing overtainting improve efficiency but miss conflicts, whereas conservative algorithms increase coverage at the cost of higher execution time and false positives. Because algorithm impact is contextual, we suggest future hybrid strategies. Such strategies could leverage the efficiency of reduced overtainting for standard cases, while applying conservative techniques to handle complex or dynamic behaviors.

VII. RELATED WORK

The challenge of detecting semantic conflicts has evolved from formal foundations to pragmatic lightweight static analysis and, more recently, to generative machine learning models. We categorize existing research into five main axes: foundational and lightweight static analysis, the impact of call-graph construction on analysis precision, machine learning for merge conflict resolution, empirical studies and characterization, and alternative detection strategies.

Foundations and Lightweight Static Analysis. The formal basis for semantic program integration was introduced by Horwitz et al. [6], who used Program Dependency Graphs (PDGs) to identify non-interfering versions. This was later extended to procedural languages using System Dependency Graphs (SDGs) [12]. While theoretically robust, these early methods rely on complete SDGs, which often lead to impractical runtimes for real-world codebases. Early tools such as Semantic Diff [28] proved hard to scale, primarily because they required building complete dependency structures for each merge scenario. In more recent work, Barros Filho [7] used Information Flow Analysis (IFC) with the JOANA framework [29] to estimate conflict between contributions in the same method. However, this implementation suffered from false positive rates (up to 57%), due to the conservative nature of the underlying call graphs. Although theoretically sound, these methods are computationally expensive, often taking hours or days to analyze large-scale Java projects. To overcome these scalability barriers, recent research focuses on lightweight static analysis applied directly to the integrated version of the code. Building on this, Santos de Jesus et al. [13] introduced four interprocedural analyses (Direct Flow, Confluence Flow, Override Assignment, and Program Dependence Graph) that run in a median execution time of 17.8 seconds per unit. While de Jesus et al. demonstrates that lightweight static analysis is feasible and provides an initial evaluation on a 99-unit manually labeled dataset (MergeDataset), it treats the call-graph algorithm as a fixed parameter, leaving the effect of call-graph algorithms on accuracy unquantified. Our work bridges this gap by isolating the effect of four distinct call-graph algorithms on these lightweight analyses. Furthermore, we expand the scope of evaluation with a larger, diverse and unlabeled dataset (RefDataset).

Call-Graph Construction and Precision. The trade-offs between precision and performance in call-graph construction are well-studied in general static analysis. Bodden [18] observed that higher precision can improve performance by

avoiding *overtainting*, the propagation of information to unreachable parts of the program. While points-to-based algorithms like Spark [14] and VTA [22] are known to reduce timeouts and prune edges [15], their specific impact on the "reachability" required for semantic conflict detection has not been quantified. Our findings align with systematic robustness evaluations [19] that highlight unsoundness in call graphs due to dynamic features like reflection. We contribute by quantifying that, for merge conflicts, the performance gains from pointer analysis often come at the cost of false negatives, making the choice of algorithm a critical decision for developers.

Machine Learning for Merge Conflict Resolution. Recent advancements in machine learning (ML) and Large Language Models (LLMs) have shifted focus towards automated conflict resolution. Zhang et al. [11] utilized GPT-3 with k-shot learning to resolve semantic conflicts in MS Edge, eliminating the need for expensive fine-tuning. Similarly, Svyatkovskiy et al. [30] and Dinella et al. [31] proposed transformer-based models like MergeBERT and DeepMerge to resolve textual and structural conflicts by rearranging or generating code tokens. Shen et al. [32] combined lightweight classifiers with ChatGPT to handle complex merge scenarios via prompt engineering, while Dong et al. [33] introduced fine-grained representations to improve generative models for conflict resolution. While these approaches show high accuracy in resolving conflicts, they often rely on existing tokens or are limited by model context sizes and potential hallucinations, whereas our work remains focused on the preceding challenge of precise detection.

Empirical Studies and Characterization. Large-scale empirical studies provide insights into the prevalence and nature of merge conflicts. Ghiotto et al. [34] analyzed over 960,000 merges in Java projects, revealing that 87% of resolutions do not require writing new code. Shen et al. [35] further characterized textual, build, and test conflicts across 208 repositories, while Jin et al. [36] explored challenges specific to Android upgrade conflicts. Recent evaluations of merge tools, such as the ConflictBench benchmark [37] and the study by Schesch et al. [38], highlight the need for rigorous testing methodologies to assess the correctness and human effort involved in resolution. These studies underscore the importance of accurate detection as a prerequisite for effective conflict management.

Alternative Detection Strategies. Other researchers explore dynamic, verification-based, and advanced static methods, which we group into three main areas. *Formal and Symbolic Methods:* SafeMerge [8] uses formal verification to prevent unwanted behaviors, but often requires restricted language subsets. Symbolic execution [39] identifies path conditions leading to conflicts in specific domains. *Test-based Methods:* Existing tools [9] rely on automated test generation to detect conflicts, while ConflictLens [40] combines LLMs with automated testing. *Build Conflicts:* Specific analyses [41], [42] target higher-order dependencies in build systems. While these methods vary in precision and scale, our approach fundamentally focuses on the impact of call-graph construction on lightweight static analysis across large-scale real-world

datasets.

VIII. CONCLUSIONS

We evaluated the effect of call-graph algorithms on the accuracy and performance of semantic conflict detection. Analyzing 12 configurations across two datasets quantified how call-graph construction influences the precision-recall trade-off.

Highly refined algorithms, particularly SPARK and VTA, reduced execution time and false positives, improving precision and overall accuracy. However, this optimization increased false negatives and sharply reduced recall compared to conservative algorithms like CHA and RTA. While precision-oriented pruning successfully filtered out spurious interferences, it masked real conflicts involving dynamic language features such as reflection.

These findings establish call-graph algorithm selection as a determinant factor for semantic conflict detection. Future work should explore context-aware hybrid algorithms that adaptively switch between conservative and refined abstractions based on local complexity, and extend this evaluation beyond Java to validate its generalizability.

REFERENCES

- [1] S. Khanna, K. Kunal, and B. C. Pierce, "A formal investigation of diff3," in *Foundations of Software Technology and Theoretical Computer Science*, 2007.
- [2] J. Clementino, P. Borba, and G. Cavalcanti, "Textual merge based on language-specific syntactic separators," in *Proceedings of the XXXV Brazilian Symposium on Software Engineering*, 2021. [Online]. Available: <https://doi.org/10.1145/3474624.3474646>
- [3] P. Accioly, P. Borba, and G. Cavalcanti, "Understanding semi-structured merge conflict characteristics in open-source java projects," *Empirical Software Engineering*, 2018. [Online]. Available: <https://doi.org/10.1007/s10664-017-9586-1>
- [4] G. Cavalcanti, P. Borba, G. Seibt, and S. Apel, "The impact of structure on software merging: semistructured versus structured merge," in *International Conference on Automated Software Engineering*, 2019.
- [5] A. Trindade Tavares, P. Borba, G. Cavalcanti, and S. Soares, "Semistructured merge in javascript systems," in *International Conference on Automated Software Engineering*, 2019.
- [6] S. Horwitz, J. Prins, and T. Reps, "Integrating noninterfering versions of programs," *ACM Trans. Program. Lang. Syst.*, 1989. [Online]. Available: <https://doi.org/10.1145/65979.65980>
- [7] R. S. M. d. BARROS FILHO, "Using information flow to estimate interference between same-method contributions," Master's thesis, Universidade Federal de Pernambuco, 2017.
- [8] M. Sousa, I. Dillig, and S. K. Lahiri, "Verified three-way program merge," in *Proceedings of the Conference on Object-Oriented Programming, Systems, Languages, and Applications*, ser. OOPSLA, 2018. [Online]. Available: <https://doi.org/10.1145/3276535>
- [9] L. D. Silva, P. Borba, W. Mahmood, T. Berger, and J. Moisakis, "Detecting semantic conflicts via automated behavior change detection," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2020.
- [10] C. Sung, S. K. Lahiri, M. Kaufman, P. Choudhury, and C. Wang, "Towards understanding and fixing upstream merge induced conflicts in divergent forks: An industrial case study," in *International Conference on Software Engineering*, 2020. [Online]. Available: <https://doi.org/10.1145/3377813.3381362>
- [11] J. Zhang, T. Mytkowicz, M. Kaufman, R. Piskac, and S. K. Lahiri, "Using pre-trained language models to resolve textual and semantic merge conflicts (experience paper)," in *International Symposium on Software Testing and Analysis*, 2022. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3533767.3534396>
- [12] D. Binkley, S. Horwitz, and T. Reps, "Program integration for languages with procedure calls," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 1995. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/201055.201056>

- [13] G. Santos de Jesus, P. Borba, R. Bonifácio, and M. Barbosa de Oliveira, "Comparing static analyses for improved semantic conflict detection," *Automated Software Engineering*, vol. 33, no. 2, pp. 1–38, 2025. [Online]. Available: <https://doi.org/10.1007/s10515-025-00580-y>
- [14] O. Lhoták and L. Hendren, "Scaling java points-to analysis using spark," in *Proceedings of the 12th International Conference on Compiler Construction*, 2003.
- [15] Y. Li, T. Tan, A. Møller, and Y. Smaragdakis, "Precision-guided context sensitivity for pointer analysis," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, p. 141, 2018. [Online]. Available: <https://doi.org/10.1145/3276511>
- [16] J. Dean, D. Grove, and C. Chambers, "Optimization of object-oriented programs using static class hierarchy analysis," in *European Conference on Object-Oriented Programming*, 1995.
- [17] V. Lira, P. Borba, R. Bonifácio, G. Santos, and M. Barbosa, "Refilter: Improving semantic conflict detection via refactoring-aware static analysis," 2025. [Online]. Available: <https://arxiv.org/abs/2510.01960>
- [18] E. Bodden, "The secret sauce in efficient and precise static analysis: the beauty of distributive, summary-based static analyses (and how to master them)," in *International Symposium on Software Testing and Analysis*, 2018. [Online]. Available: <https://doi.org/10.1145/3236454.3236500>
- [19] M. Reif, "Novel approaches to systematically evaluating and constructing call graphs for java software," Ph.D. dissertation, Technische Universität Darmstadt, 2021.
- [20] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: a java bytecode optimization framework," in *CASCON First Decade High Impact Papers*, 2010. [Online]. Available: <https://doi.org/10.1145/1925805.1925818>
- [21] D. F. Bacon and P. F. Sweeney, "Fast static analysis of c++ virtual function calls," in *Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 1996. [Online]. Available: <https://doi.org/10.1145/236337.236371>
- [22] J. Späth, K. Ali, and E. Bodden, "Ideal: efficient and precise alias-aware dataflow analysis," *Proceedings of the ACM on Programming Languages*, 2017. [Online]. Available: <https://doi.org/10.1145/3133923>
- [23] T. A. Ghaleb, S. Hassan, and Y. Zou, "Studying the interplay between the durations and breakages of continuous integration builds," *IEEE Transactions on Software Engineering*, 2023.
- [24] "Online appendix," available at: <https://anonymous.4open.science/w/papers-AEC8/the-effect-of-pointer-analysis-for-semantic-conflict-detection.html>, 2025.
- [25] K. Karakaya, P. Muthuraman, and E. Bodden, "Pick your call graphs well: On scaling ifds-based data-flow analyses," in *Proceedings of the 14th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, 2025. [Online]. Available: <https://doi.org/10.1145/3735544.3735587>
- [26] D. Landman, A. Serebrenik, and J. J. Vinju, "Challenges for static analysis of java reflection - literature review and empirical study," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, 2017.
- [27] V. Kanvar and U. P. Khedker, "Heap abstractions for static analysis," *ACM Comput. Surv.*, 2016. [Online]. Available: <https://doi.org/10.1145/2931098>
- [28] Jackson and Ladd, "Semantic diff: a tool for summarizing the effects of modifications," in *Proceedings 1994 International Conference on Software Maintenance*, 1994.
- [29] J. Graf, M. Hecker, and M. Mohr, "Using joana for information flow control in java programs — a practical guide," *Lecture Notes in Informatics (LNI), Proceedings - Series of the Gesellschaft für Informatik (GI)*, 2013.
- [30] A. Svyatkovskiy, S. Fakhoury, N. Ghorbani, T. Mytkowicz, E. Dinella, C. Bird, J. Jang, N. Sundaresan, and S. K. Lahiri, "Program merge conflict resolution via neural transformers," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022. [Online]. Available: <https://doi.org/10.1145/3540250.3549163>
- [31] E. Dinella, T. Mytkowicz, A. Svyatkovskiy, C. Bird, M. Naik, and S. Lahiri, "Deepmerge: Learning to merge programs," *IEEE Trans. Softw. Eng.*, 2023. [Online]. Available: <https://doi.org/10.1109/TSE.2022.3183955>
- [32] C. Shen, W. Yang, M. Pan, and Y. Zhou, "Git merge conflict resolution leveraging strategy classification and llm," in *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, 2023.
- [33] J. Dong, Q. Zhu, Z. Sun, Y. Lou, and D. Hao, "Merge conflict resolution: Classification or generation?" in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 2024. [Online]. Available: <https://doi.org/10.1109/ASE56229.2023.00155>
- [34] G. Ghiotto, L. Murta, M. Barros, and A. van der Hoek, "On the Nature of Merge Conflicts: A Study of 2,731 Open Source Java Projects Hosted by GitHub," *IEEE Transactions on Software Engineering*, 2020.
- [35] B. Shen, M. A. Gulzar, F. He, and N. Meng, "A characterization study of merge conflicts in java projects," *ACM Transactions on Software Engineering and Methodology*, 2023. [Online]. Available: <https://doi.org/10.1145/3546944>
- [36] W. Jin, M. Sun, J. Zhou, J. Shang, Z. Huang, and T. Liu, "Software architecture matters: Challenges and opportunities for android upgrade conflicts in practice," *ACM Trans. Softw. Eng. Methodol.*, 2026. [Online]. Available: <https://doi.org/10.1145/3744921>
- [37] B. Shen and N. Meng, "Conflictbench: A benchmark to evaluate software merge tools," *J. Syst. Softw.*, 2024. [Online]. Available: <https://doi.org/10.1016/j.jss.2024.112084>
- [38] B. Schesch, R. Featherman, K. J. Yang, B. Roberts, and M. D. Ernst, "Evaluation of version control merge tools," in *International Conference on Automated Software Engineering*, 2024. [Online]. Available: <https://doi.org/10.1145/3691620.3695075>
- [39] W. Muylaert, J. Härtel, and C. De Roover, "Symbolic execution to detect semantic merge conflicts," in *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2023, pp. 186–197.
- [40] L. Sun, Y. Lu, X. Mao, T. Zhang, Z. Zhang, and H. Zhou, "Conflictlens: An llm-based method for detecting semantic merge conflicts," in *Proceedings of the 37th International Conference on Software Engineering and Knowledge Engineering (SEKE 2025)*, 2025. [Online]. Available: <https://ksiresearch.org/seke/seke25paper/paper012.pdf>
- [41] S. S. Towqir, B. Shen, M. A. Gulzar, and N. Meng, "Detecting build conflicts in software merge for java programs via static analysis," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2023. [Online]. Available: <https://doi.org/10.1145/3551349.3556950>
- [42] L. Da Silva, P. Borba, and A. Pires, "Build conflicts in the wild," *Journal of Software: Evolution and Process*, 2022. [Online]. Available: <https://doi.org/10.1002/smr.2441>